

Garry Tan 複雜度棘輪 繁中精解

Garry Tan：複雜度棘輪 × 90% 測試覆蓋率（繁體中文精解版）

來源：[@garrytan 推文 2054064931515855118](#) 譯註版本：v1.0 · 2026-05-19 · vins-hub/vinshub
授權：以 CC BY-SA 4.0 釋出

一句話濃縮

AI agent 寫程式比人類快太多。瓶頸從「生產」搬到「驗證」。在 2026-2028 這 24 個月內，能贏的不是 agent 最快的公司，而是 guardrail 最強的公司——具體形式：把測試覆蓋率從 80% 拉到 90%。

“The companies that win the next 24 months won’t be the ones with the fastest agents...They’ll be the ones with the strongest guardrails.” — Garry Tan (Y Combinator CEO)

為什麼這條改變了一切

過去 20 年，軟體業的瓶頸是「寫不夠快」。所以工程師最值錢的能力是寫程式速度、設計能力、解難能力。

2025 起，AI agent 已能 1 小時寫完人類 1 週的程式碼。瓶頸不再是寫，是「確保它沒寫壞」。

這就是 Garry Tan 命名的「複雜度棘輪（Complexity Ratchet）」：

每部署一個 feature，就增加依賴與 edge case，這些越來越難追蹤。當 AI agent 比人類快 10 倍時，這個棘輪轉得遠超人類驗證能力。結果：你的 **codebase** 累積技術債的速度首次超過了人類有可能審查的速度。

Garry 的處方：把「90% 自動化測試覆蓋率」當作組織級的新基線。

這份學習材料把這個處方拆解成可執行步驟。

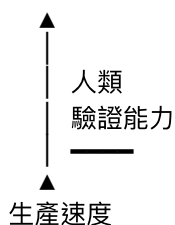
心智模型

AI Agent 時代之前

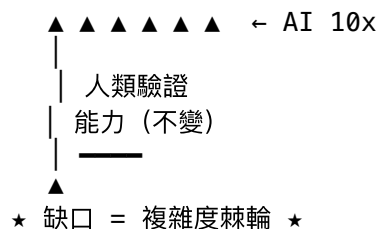
生產速度

AI Agent 時代之後

生產速度



瓶頸：寫不夠快
解法：請更多工程師



瓶頸：審不夠快
解法：90% 自動化測試 +
AI 寫測試

核心三問 (CEO / Tech Lead 必答)

Garry 給組織領導層三個必須清楚回答的問題：

1. **What is our verification coverage on AI outputs?** 「我們對 AI 輸出的驗證覆蓋率是多少？」
2. **Where does the ratchet bite hardest — and is that where we deployed our first agent?** 「複雜度棘輪在哪裡咬最深？我們把第一個 agent 部署在那邊嗎？」
3. **Who owns catching regressions before they reach customers/citizens/patients?** 「誰負責在 regression 影響到客戶 / 民眾 / 病患之前抓到它？」

如果你公司沒有人能 5 秒內回答這三題，你已經被棘輪追上。

為什麼是 90% (不是 80%、不是 100%)

覆蓋率	評估
< 60%	不能上 production。任何 PR 都可能 silent breakage
60-79%	過去 10 年業界中位數。AI 時代不夠
80%	舊典範的 best practice 。人類維護 codebase 的甜蜜點
90%	AI 時代的新基線 。AI 寫測試 ~= 免費，所以阻力消失
95-100%	Diminishing returns。維護 cost 超過邊際效益。少數高風險系統值得（醫療 / 金融 / 太空）

Garry 的論點：過去 90% 不普及不是因為它技術上難，而是因為「人類寫測試的意志力很貴」。AI 把這個成本壓到接近零後，**90%** 從「奢侈」變成「應有」。

學習路徑

章節	內容	適合誰
01. 複雜度棘輪	核心概念 + 為何 AI 時代是 inflection point	所有工程 leader
02. 驗證瓶頸	從「production bottleneck」到「verification bottleneck」	戰略層
03. 為何 90% 是新基線	數學論證 + AI 經濟學論證	想說服老闆的人
04. 三大組織問題	三問的展開與診斷工具	CEO / VPE / EM
05. 測試類型矩陣	Unit / Integration / E2E / Property / Mutation / Fuzz	Tech Lead
06. AI 當測試作者	用 AI 寫測試的具體 SOP	工程師
07. 反模式：假測試陷阱	80% 覆蓋率 ≠ 80% 品質	避免騙自己
08. 12 週實施路線圖	從 65% → 90% 的階段計畫	想實際執行的人
09. 跟 12-factor agents 的關聯	Verification 是 agent 工程化的必要條件	系統派

30 秒急救包

如果你現在沒時間讀全部，做這 3 件事：

1. 查現狀：你的 main repo 跑 coverage report，看數字。如果 < 80%，這是你的第一個 sprint 目標
2. 訂目標：在 CI 加 --fail-under=85（從現在的數字+5%），逐月拉到 90%
3. 指派 **owner**：選一個 Tech Lead 全權負責測試品質，賦予阻擋 PR 的權限

剩下的可以慢慢讀。

為什麼這份指南對 2026 的工程組織重要

1. **AI agent** 平均產出已超越中階工程師：寫 code 不再是 bottleneck
2. 大多數公司還用 **2018-2023** 的 **testing playbook**：80% coverage、手動 review、定期 audit
3. 複雜度棘輪在 **6-12** 個月內咬死你：當 incident 多到讓 SRE 加班，已經晚了

Garry Tan 的方法論不是 nice-to-have，是 **defensive moat for the next 24 months**。

譯註者觀察

Garry 這條推文之所以重要，是因為他是少數同時看 YC 上百家新創 + 自己用 AI 大量寫程式的人。他看到的不是理論，是 portfolio 公司的真實發病。

他的處方有個特點：便宜、可量化、可立即執行。不需要新工具、不需要新框架、不需要組織重構——只需要把 **coverage** 數字當作 **production** 指標。

這份指南把這個處方擴展成 12 週可執行路線圖。

接下來

[Chapter 01: 複雜度棘輪 — 為何 AI 時代是 inflection point](#)

[← 回 README](#)

第 01 章：複雜度棘輪 — 為何 AI 時代是 Inflection Point

核心句：每個 feature 部署，都把 codebase 的複雜度往上推一格、且鎖住不能往下——這就是 ratchet（棘輪）。AI agent 把這個棘輪轉動速度提高 10 倍，人類驗證能力不變，缺口就是當前最致命的工程風險。

什麼是 ratchet（棘輪）

棘輪是個機械零件：可以往一個方向轉，但有齒輪鎖住，不能反向。

軟體裡的「複雜度棘輪」意思：

動作	ratchet 方向	為何不能反向
部署一個新 feature	複雜度 +1	用戶開始依賴，移除 = breaking change
加一個 API endpoint	依賴關係 +1	下游服務開始呼叫，下架要協調
加一張 database 表	schema 複雜度 +1	開始有資料、開始有 query、開始有 ORM mapping
加一個 config flag	配置矩陣 $\times 2$	兩個值都得測試
加一個 conditional branch	程式路徑 $\times 2$	兩條路徑都得驗證

這些「+1」單個看都不大，但累積起來就是指數爆炸。一個 100K 行的 codebase 可能有 2^{30} 條可能的 program path——人類根本不可能全部驗證。

AI Agent 加速棘輪

過去：

Human dev 一週寫 1 個 feature → 每週棘輪 +1
Human reviewer 一週審 1-2 個 feature → 跟得上

AI agent 時代：

AI agent 一天寫 5 個 feature → 每天棘輪 +5

Human reviewer 一週還是審 1-2 個 → 跟不上

Gap = (5×5 - 2) = 23 個 feature 的「未驗證債」每週累積。

3 個月後，你 codebase 有 ~300 個未充分驗證的 feature 在 production。這就是 **ratchet bite hardest** 的時刻。

為何不能「先快後審」

❌ 錯誤直覺：「先讓 AI 衝產量，之後再補 review。」

問題：1. 棘輪鎖住：上線後用戶開始用，補 review 發現問題 → fix 變成 breaking change → coordination cost 暴增 2. 複合債務：未驗證 feature 之間互相依賴，越補越亂 3. **Onboarding** 災難：新工程師看不懂 codebase 4. **Incident** 雪崩：到某個臨界點，每天都有事故

✅ 正解：驗證能力先於產出能力。先把驗證 baseline 拉高，再讓 AI agent 全速產出。

棘輪在哪些地方咬最深？

Garry 的觀察（從 YC portfolio 中常見的失敗模式）：

1. 業務邏輯層 (business logic)

- AI 改了 pricing 計算，沒人發現
- AI 改了 permission check，安全洞
- AI 改了狀態機 transition，產生不可達狀態

2. 資料 schema 變遷

- AI 加 column 但忘記 migration
- AI 改 column type 但下游 service 沒同步
- AI 加 index 但忘了 production-scale 影響

3. 整合層 (integrations)

- AI 改 API contract 但 client 沒同步
- AI 加 webhook 但沒處理 retry/idempotency
- AI 加 timeout 但 ripple effect 到其他 service

4. 配置與 feature flag

- AI 加 feature flag 但忘了清除舊路徑
- AI 改預設值但沒測試非預設值情境
- AI 加 env var 但 docs / staging / prod 不同步

5. 「無聲失敗」場景

- AI 加 error handling 但 silent swallow exception
- AI 加 retry logic 但無限循環
- AI 加 cache 但 invalidation 邏輯錯

這 5 個區域就是「**ratchet bite hardest**」的具體位置。你公司部署第一個 **AI agent**，應該優先針對其中之一加強驗證——不是隨機選用例。

Inflection Point 的判斷指標

什麼時候你公司「進入 **ratchet** 困境」？看這幾個信號：

信號	意義
Bug count 連續 3 個月上升	棘輪轉得比 fix 快
Incident 平均解時間（MTTR）變長	codebase 已經沒人完全理解
新人 onboarding > 3 個月才能 first deploy	複雜度超過合理範圍
PR review 平均 > 48 小時	reviewer 跟不上
「不敢動的 module」list 變長	technical debt 累積
Senior engineer 開始 burnout	驗證壓力都壓在他們身上

3 個以上信號出現 → 立即啟動 **ratchet defense** ([Ch 08 路線圖](#))。

反直覺洞察：限速 AI agent 是錯的

很多 leader 看到 ratchet 問題，本能反應是「那就限制 **AI** 寫程式速度」。

錯。理由：

1. 競爭對手不會限速——你限了你輸
2. **AI** 寫程式的價值就在速度——限速等於放棄價值
3. 限速不解決問題——只是把棘輪轉慢、最終還是會咬

正確反應：升級驗證能力。讓驗證速度也 10x。

怎麼讓驗證 10x？答案是 [Ch 06](#)：用 **AI** 寫測試。

棘輪轉得多快，驗證就要多快。AI 加速生產 → AI 加速驗證。唯一不可加速的是「人類做最終 **judgment**」，所以人類的 judgment 要被用在最高槓桿的地方。

棘輪 vs Technical Debt 的差別

維度	Technical Debt	Complexity Ratchet
來源	人類 shortcuts	AI 過量產出
速度	線性累積	指數累積
可逆性	可重構還清	鎖住、不可逆
主要表徵	程式碼變醜	行為變不可預測
處理方式	sprint 預算還債	驗證能力升級

Tech debt 是「程式品質下降」；ratchet 是「驗證能力被超車」。兩者都該管，但 ratchet 更危險，因為它在 **production** 才現形。

本章小結

觀念	為何重要
棘輪是「上得去下不來」的單向結構	解釋為何複雜度只增不減
AI agent 加速棘輪 10x	看清為何過去的 testing playbook 不夠
5 個 bite hardest 區域	知道把第一道防線蓋在哪
6 個 inflection point 信號	自我診斷
限速 AI 是錯的	正確的反應是升級驗證能力
ratchet ≠ tech debt	兩個概念分開管

接下來

 [Chapter 02: 驗證瓶頸 — 從 production 搬到 verification](#)

[← 回 README](#) · [← Ch 01](#)

第 02 章：驗證瓶頸 — 從 Production 搬到 Verification

核心句：軟體工程史上的瓶頸，從來都是「做不夠快」。AI agent 把這個瓶頸打破，新的瓶頸是「驗證做的東西」。看清這個轉移，你才能配置正確的資源。

60 年瓶頸演進史

時代	瓶頸	解法
1960s	記憶體 / 計算力	摩爾定律
1980s	UI / 介面複雜	圖形介面、framework
2000s	Web scale	雲端、分散式系統
2010s	開發速度	DevOps、CI/CD、microservices
2020-2024	上市時間	敏捷、API economy、SaaS
2025+	驗證 / 確認	?

過去 5 代瓶頸都在「做的快慢」這條軸上。**2025** 起，這條軸的瓶頸消失了——AI agent 可以一天寫完過去一個團隊一週的東西。

新瓶頸是另一條軸：「確認做的東西有沒有錯」。這條軸 60 年來幾乎沒進步。

為何驗證能力沒同步進步

3 個歷史原因：

原因 1：驗證本來就比寫程式難

寫程式：「讓電腦做某件事」——目標明確，工具豐富
驗證：「確認電腦做的事符合所有可能情境」——目標模糊，工具有限

證明軟體正確性是電腦科學「最難」的問題之一（Halting problem 等）。

原因 2：人類驗證的瓶頸在「注意力」不在「技能」

寫 1000 行 code：訓練可以加速
讀 1000 行 code：注意力極限是物理限制，訓練幫助有限

一個 senior engineer 在 8 小時內，認真 review 程式碼的 cap 是 ~500 LOC（業界研究）。這個數字 30 年沒變。

原因 3：測試不是新東西，但測試思維沒升級

業界從 1980s 就有測試。但測試在絕大多數組織裡仍被當作「寫完 code 之後的補課」，而不是「設計時的一階考量」。

瓶頸轉移的具體表現

表現 1：Code review 從「審美」變「續命」

過去：reviewer 看 code 主要是「這寫得好不好看」
現在：reviewer 看 code 是「這會不會壞 production」

當你的同事一週塞 50 個 PR 給你（因為他有 AI 幫忙），你連看完都做不到，更別說「審美」。

表現 2：Bug 從「漏掉」變「審不過來」

過去：bug 之所以漏掉，是因為沒寫到測試 現在：bug 漏掉，是因為寫了測試但沒 review、或 review 沒看出測試本身有問題

表現 3：Senior engineer 變成「人類 oracle」

過去：senior 寫架構、寫核心邏輯 現在：senior 主要工作變成「確認 AI 做的事對不對」

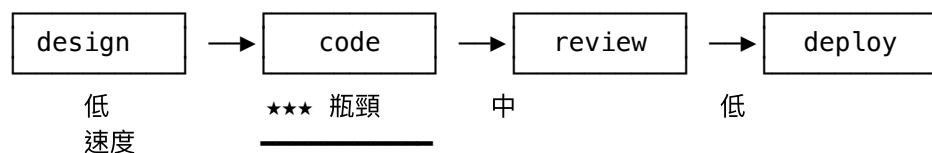
如果你公司有 5 個 senior 工程師，AI 讓他們的「生產時間」減少 50%，但「驗證 AI 產出」吃掉省下的 50% 還不夠。

表現 4：On-call 變得不可預測

過去：on-call 大致知道哪些系統不穩定 現在：incident 來自「昨天 AI 改的某行 code」，找出來都要 1 小時

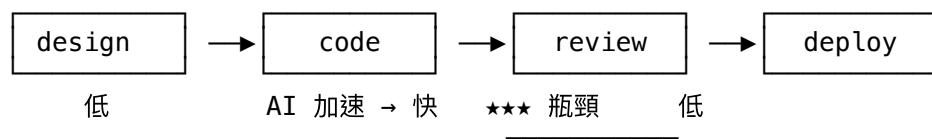
Garry Tan 的論述：把瓶頸移轉視覺化

2010s 模型：



解法：請更多工程師、用更好的工具、improve 開發流程

2025+ 模型：



解法：升級驗證能力—測試自動化 + AI 寫測試 + 90% coverage

驗證的 3 個層次

「驗證」不是單一動作，是 3 個層次的組合：

層次 1：Static verification（靜態驗證）

- Type checking (TypeScript / Mypy)
- Linter (ESLint / Pylint / Ruff)
- Static analysis (CodeQL / Semgrep)
- AI code review (自動)

特性：cheap、fast、cover broad，但抓不到行為錯誤。

層次 2：Test verification (測試驗證)

- Unit tests
- Integration tests
- E2E tests
- Property-based tests
- Mutation tests

特性：抓行為錯誤，但只能抓「你想到要測的」case。

層次 3：Production verification (生產驗證)

- Feature flag 漸進放量
- Canary deployment
- Synthetic monitoring
- Real user monitoring
- Anomaly detection

特性：抓「實際使用中的問題」，但代價是讓 user 當白老鼠。

Garry 的論點：3 個層次都要做，但**90%** 測試覆蓋率（層次 2）是新基線——因為它最能匹配 AI 的產出速度。

不是不要 review，而是 review 要被 amplify

「**90%** 覆蓋率」不是叫你不要 code review。而是把 **reviewer** 注意力解放出來做更高槓桿的事：

沒高覆蓋率時，**reviewer** 要做 有高覆蓋率時，**reviewer** 可以做

看每行 code 有沒有 bug

看 architecture 對不對

跑各種 case 在腦中模擬

看 design 假設合不合理

擔心 edge case 沒處理

擔心 product 邏輯對不對

「這會不會壞 X 功能」

「這個 feature 該不該做」

測試是 **reviewer** 的 **force multiplier**。Reviewer 不需要再扮演「人類 **oracle**」，他可以扮演「戰略 **partner**」。

反論：「**100% coverage** 也救不了」

有人會反駁：「就算 100% line coverage，也不保證對。」

對。但這不否定「90% 比 60% 好」這件事。

關鍵區別：- **line coverage**：每行至少被執行一次（可能根本沒 assert）- **branch coverage**：每個 if-else 兩支都跑過 - **mutation score**：故意改 code，看測試會不會抓到（這才是真品質指標）

[第 05 章](#) 會展開測試類型矩陣。**Garry** 講的 90% 是「有意義」的 90%，不是「為達標而達標」的 90%。

對組織的意涵

當你看清「瓶頸已轉移」這件事，這些 decisions 變得清楚：

Decision	過去做法	新做法
Hire 順位	多 hire developer	多 hire 測試 / SRE / 平台工程師
預算分配	70% feature dev / 30% infra	50% / 50%
Sprint 規劃	feature 為主	feature + verification capacity 雙軌
Code review SLA	24 小時內	「測試 + AI 預審先過、人類 review 48 小時內**
Senior 角色	寫 hard problem 的 code	設計 testability、設計 verification 系統

本章小結

觀念	為何重要
60 年瓶頸演進，從速度到驗證	看清歷史脈絡
驗證能力沒同步進步的 3 個原因	知道對手在哪
瓶頸轉移的 4 個具體表現	自我診斷組織狀態
3 個驗證層次（static / test / prod）	配置完整防線
測試是 reviewer 的 force multiplier	改變 reviewer 角色
90% 覆蓋率 ≠ 100% 對	但 90% 仍然遠優於 60%

接下來

[➡ Chapter 03: 為何 90% 是新基線](#)

[← 回 README](#) · [← Ch 02](#)

第 03 章：為何 90% 是新基線

核心句：80% 是「人類意志力天花板」造成的歷史均衡點。AI 把寫測試的成本降到接近零後，這個天花板消失了。新的均衡點是 90%——不是因為 90% 完美，而是因為 80% 已經不夠應付 AI 加速的棘輪。

80% 從哪裡來

80% 不是某個科學論證得出的數字，而是 30 年來經驗加總的「比較划算的點」。

80% 的歷史定位

覆蓋率區間	經驗判斷
< 50%	太少，任何改動風險都很高
50-70%	基本面，但 critical path 還會漏
70-85%	甜蜜點：抓大部分 bug，維護 cost 可承受
85-95%	Diminishing returns 開始
> 95%	維護 cost 超過 marginal value，除了高風險系統

業界中位數一直在 60-70%，少數高品質組織能到 80-85%。**90%** 是「奢侈品」——因為人類寫測試實在太貴了。

80% 的隱形假設

80% 之所以是甜蜜點，前提是「寫測試的人類意志力」是稀缺資源。具體：

- Senior engineer 一天能寫 5-10 個高品質測試
- 寫測試比寫 feature 無聊，工程師不喜歡寫
- TDD 雖然有效但對新人有上手成本
- 寫測試 ROI 不明顯（看不到「這個測試救了我多少次」）

這些前提讓「多寫 10% 測試」要付出「Sr engineer 一個月」的代價。對中型公司來說，「從 80% 拉到 90%」需要兩個 quarter，這個 budget 通常被 feature work 排擠。

AI 怎麼改變這個方程式

AI 寫測試 ≈ 免費

當 Claude / GPT-4 / Cursor 等工具進入工程流後：

動作	人類時間	AI 時間
為一個 function 寫 unit test	15-30 分鐘	30 秒
為一個 class 寫完整測試套	2-4 小時	5 分鐘
為一個 module 寫 integration test	4-8 小時	15 分鐘
補一個邊角 case 測試	10 分鐘（注意力切換）	30 秒

AI 把「寫測試的成本」壓到接近零。

那「80% 是甜蜜點」的論證還成立嗎？

不。原本 80% 是甜蜜點是因為：

marginal cost of writing test $\approx$ marginal value at 80%

當 cost 接近 0：

marginal cost ≈ 0
marginal value at 90% > 0

$\therefore$ 90% 變新均衡

AI 寫測試的具體經濟學

假設你有一個 100K 行的 codebase，目前 75% coverage：

情境	人類拉到 90%	AI 輔助拉到 90%
需要新增測試	~5000 個測試	同
Senior engineer 時間	1250 小時（~7 個月 FTE）	100 小時 review + AI 寫
直接成本	\$200K（按 \$160/hr）	\$20K（含 LLM API cost）
Calendar 時間	6 個月	3 週
機會成本（沒做 feature）	6 個月 product roadmap	3 週 product roadmap

結論：用 AI 寫測試，從 75% 拉到 90% 在 3 週、\$20K 內可完成。對 Y Combinator 規模的新創，這完全在預算內。

90% 的具體 benchmark

「90% coverage」這句話太抽象，必須拆解：

分項目標

維度	90% 基線含意
Line coverage	90% 的程式行被測試執行過

維度	90% 基線含意
Branch coverage	90% 的 if-else 兩支都被測過
Function coverage	90% 的函式有 ≥ 1 個測試
Critical path coverage	核心業務流程 100% 覆蓋
Mutation score	$\geq 75\%$ (mutation testing 抓住假測試)

單看 **line coverage** 達到 **90%** 不夠。Garry 的 **90%** 是綜合性的：core path 100% + branch $\geq 90\%$ + mutation $\geq 75\%$ 。

分系統優先級

不是所有 module 都需要 90%，但 critical 系統必須：

系統類型	覆蓋率基線
Payment / billing / auth	95-100%
核心業務邏輯	90-95%
資料 schema / migration	90-95%
API contract	90%
UI / 前端	70-85% (用 E2E 補)
Build script / 內部工具	50-70%
Glue code / config	不強求

90% 是「aggregate weighted」，不是每個檔案都 90%。

90% 不是終點：90% + 還要什麼？

達成 90% 覆蓋率只是充分條件之一。完整防線還需要：

1. 測試品質保證

防止「假測試」：見 [Ch 07 反模式](#)。具體：- Mutation testing (Stryker / mutmut) - AI 評審測試品質 ([Ch 06](#)) - 拒絕 assert True 類的 placeholder

2. CI 防線

- `--fail-under=90` 在 CI 強制
- PR 不能降低覆蓋率 (diff coverage 必須 $\geq 90\%$)
- 不能 disable 測試 (除非 marked 並有清楚 expiry)

3. Observability 補測試漏網

- Error tracking (Sentry)

- Performance monitoring (Datadog)
- Anomaly detection
- User behavior analytics

4. Postmortem 文化

每個 production incident → 「為何測試沒抓到？」 → 補測試 → 防 regression

反論與回應

反論 1：「90% coverage 還是會有 bug」

回應：對，但 90% bug 比 60% bug 少 4 倍以上（業界經驗）。沒有完美防線，只有性價比更高的防線。

反論 2：「coverage 不等於品質」

回應：完全正確。所以才有 [Ch07 反模式](#) 跟 mutation testing。Garry 的 90% 是有意義的 90%，不是 gaming。

反論 3：「我們是 startup，沒時間寫測試」

回應：這是過時思維。現在寫測試的時間，是過去的 1/10。「沒時間寫測試」基本上等於「不會用 AI」。

反論 4：「AI 寫的測試不可靠」

回應：AI 寫的測試需要 human review，但人類 review 一個測試 < 寫一個測試的 1/10 時間。槓桿仍然成立。

反論 5：「我們有 manual QA」

回應：Manual QA 規模化不了。Manual QA 抓的 bug 數量是線性，AI 產出 bug 數量是指數。數學上贏不了。

為什麼是 90%，不是 92% 或 88%？

90 是個 round number，溝通方便。但實際真正的閾值：

- **Critical path**：100%（不容妥協）
- **Business logic**：95%（極少例外）
- **Aggregate**：90%（CI gate 的數字）
- **UI / 邊緣**：85%（E2E 補）

90% 作為組織級宣示，是個容易理解、易執行、易檢核的數字。「今年要 90%」比「今年要綜合品質指標達到 P95」好溝通 10 倍。

跟業界其他 benchmark 對照

標準	數字	對應
Google internal	60-70% line	過時了
Meta internal	70-80%	接近
FAANG critical service	85-90%	對
醫療軟體 (FDA)	95-100%	高於 90%
航太軟體 (NASA)	100% MC/DC	最高標準
Garry Tan AI 時代提議 90% (with quality) 新基線		

90% 已經不算激進，只是把過去 FAANG critical service 的標準普及到所有 AI-augmented 組織。

本章小結

觀念	為何重要
80% 是人類意志力造成的天花板	解釋為何過去不該 push 更高
AI 把寫測試成本壓到接近零	解釋為何現在該 push 更高
90% 是「aggregate weighted」	不是每個檔案都 90%
90% ≠ 完美	還需要品質檢核、CI 強制、observability
5 個反論的回應	對抗組織內阻力
90% 是 critical service 早就在做的	不激進，只是普及

接下來

[➡ Chapter 04: 三大組織問題 — 三問的展開與診斷工具](#)

[← 回 README](#) · [← Ch 03](#)

第 04 章：三大組織問題 — 三問的展開與診斷工具

核心句：Garry Tan 的三大問題不是修辭，是診斷工具。任何組織只要 5 秒內回答不出來，就已經被棘輪追上。本章把這三問展開成可量化、可執行的組織體檢表。

三問原文

- 1. What is our **verification coverage** on AI outputs?
- 2. Where does the **ratchet bite hardest** — and is that where we deployed our first agent?
- 3. **Who owns** catching regressions before they reach customers/citizens/patients?

每一問都對應組織的一個 critical 維度：

問	維度	失敗後果
Q1 能力：你有沒有看 AI 產出的能力	不知道 AI 在做什麼	
Q2 戰略：你把資源放在哪	棘輪在 A，agent 在 B，白做	
Q3 究責：誰負責	沒人 own → 永遠救火	

Q1：What is our verification coverage on AI outputs?

字面拆解

Verification coverage on AI outputs ≠ **code coverage**。Garry 問的不只是「多少行被測試到」，是完整的驗證鏈：

層次	對 AI 產出的驗證
L1：寫進 PR	有沒有 type check、lint、static analysis pass？
L2：自動測試	有沒有 unit / integration / E2E 跑過？
L3：人類審	有沒有 senior 看過邏輯與設計？
L4：staging	有沒有在類似 prod 環境跑過？
L5：canary	有沒有用真實流量小規模驗證？
L6：監控	Production 有沒有監控覆蓋？

Verification coverage = 6 個層次都做到的比例。

診斷表：你的 **verification coverage** 真實分數

填這個表（誠實）：

維度	目標	你目前	gap
L1: 100% PR 通過 lint + type check	100%	__%	__
L2: aggregate test coverage	90%	__%	__
L2: mutation score	75%	__%	__
L2: critical path test coverage	100%	__%	__
L3: 100% PR 有人類 reviewer	100%	__%	__
L3: 平均 review SLA	< 24h	__h	__
L4: PR 必先過 staging	100%	__%	__
L5: prod 部署用 canary / feature flag	100%	__%	__
L6: critical service 有 alert	100%	__%	__

任何維度 < 80% 都是「verification 缺口」。

Q1 的隱含結論

如果你回答不出每個維度的數字，你的組織連自己的 **verification gap** 都不知道。第一步是測量：用 [Ch 08 路線圖](#) 的 Week 1-2 行動。

Q2 : Where does the ratchet bite hardest — and is that where we deployed our first agent?

字面拆解

兩個子問題：1. 棘輪在哪裡咬最深？（識別風險區）2. 你的第一個 AI agent 在這裡嗎？（驗證部署優先級）

如果 1 跟 2 不匹配，你的 **AI** 部署戰略錯了。

找出 **ratchet bite hardest** 的方法

3 個資料源：

資料源 1 : Incident 歷史

過去 12 個月所有 P0 / P1 incident，按系統分類：

系統 A: 8 次 P1
系統 B: 3 次 P1
系統 C: 12 次 P1 ← bite hardest
系統 D: 1 次 P1
系統 E: 5 次 P1

C 是 ratchet 咬最深的地方——bug 一直冒。

資料源 2 : MTTR (Mean Time To Recovery)

系統 A: 平均 30 min
系統 C: 平均 3 hours ← 不只 bug 多，而且難修
系統 D: 平均 15 min

C 同時兼具「**bug 多 + 難修**」——典型 ratchet 困境。

資料源 3 : 「不敢動」清單

問 senior engineers：「哪些 module 你不敢改？」答案會集中在 1-3 個 module。那就是 **ratchet bite hardest** 的地方。

Q2 的具體 action

將 3 個資料源交集，挑出 top 3 ratchet 區。你的第一個 AI agent 應該部署在這裡——但是部署成「測試生成 agent」，不是「feature 生成 agent」。

順序	你做的事
第 1 個 agent	為 top 1 ratchet 區寫測試的 agent
第 2 個 agent	為 top 2 ratchet 區寫測試的 agent
第 3 個 agent	為 top 1 區寫 monitoring / alert 的 agent
之後	才開始 feature-writing agent

反直覺：別人都在用 AI 寫 feature，你應該先用 AI 補防線。

Q3 : Who owns catching regressions before they reach customers?

字面拆解

Who owns 是 organizational accountability。「測試是大家的事」=「沒人負責」。

三種 ownership 模式

模式	結構	問題
Distributed（散落）	每個工程師寫自己的測試	無人總攬，品質參差，棘輪悄悄轉
Centralized（集中）	QA 團隊負責所有測試	開發跟測試脫節，QA 反應慢，瓶頸
★ Hybrid（混合）	每個 PR 作者寫測試 + 中央 platform team 設標準與工具	規模化、有 oversight

Garry 推 Hybrid。具體形式：

角色設計

[Engineering team]	[Platform / Quality team]
↓	↓
寫 feature code	設計測試 framework / fixture
寫 first-pass tests	設計 CI gate 標準
跑 local tests	負責 mutation testing infra
PR submit	跑 nightly extensive test suite
	設計 verification SOPs
	負責 80% → 90% migration
	負責 AI test generation 工具
	負責 metric 收集與報告
	★ HAS POWER TO BLOCK ★

關鍵點：Platform/Quality team 必須有阻擋 **PR** 的權力。沒這權力，他們建議會被無視。

Q3 的具體 action

3 個 hire / appoint 任務：

Hire	職責	Headcount（每 50 工程師）
Quality Engineer	設計測試標準 / 工具 / fixture	1-2 個
Verification Architect	設計 verification system（CI / mutation / canary）	1 個
Reliability lead	連結 verification 跟 production reliability	1 個

這些不是新角色，他們以前都叫“QA”、“Build engineer”、“SRE”。Garry 的洞察：在 AI 時代，這些角色從「second-class」變成「first-class」——薪水、地位、影響力都要跟 senior dev 持平。

三問檢核表

填這個表，每題 0-10 分（10 = 完全做到）：

問題	子項	分數
Q1：verification coverage L1-L6 6 個層次平均		___/10
Q1	知道每個系統的 coverage 數字	___/10
Q1	有 mutation testing	___/10
Q2：知道 ratchet 在哪	有 incident 歷史分析	___/10
Q2	有 MTTR 數據	___/10
Q2	第一個 AI agent 部署在 ratchet 區	___/10
Q3：有人 own	有 quality team	___/10
Q3	Quality team 有阻擋 PR 權	___/10
Q3	Senior 級別的 verification ownership	___/10

總分判讀：

區間	評估
80-90	你已進入 AI 時代
60-79	還在追趕，加速
40-59	棘輪在咬你，急救
< 40	緊急狀態，立即啟動 Ch 08 路線圖

三問背後的世界觀

這三問的共同精神：

AI 不是省人力工具，是 **leverage** 放大鏡——你過去 **strong** 的地方會被放大，**weak** 的地方也會被放大。

過去 verification 弱的組織，AI 時代會被棘輪咬死。過去 verification 強的組織，AI 時代會把優勢放大成壟斷。

這是「未來 24 個月誰贏」的本質。

給 CEO / CTO 的 1 分鐘版本

如果你只有 1 分鐘跟 CEO 簡報這章，講這個：

「老闆，我們有三個必須回答的問題：

- (1) 我們對 AI 產出的驗證覆蓋率是多少？——如果 5 秒內答不出來，我們已經輸了。
- (2) 我們最大的技術風險在哪？我們的第一個 AI 部署在那邊嗎？——如果 agent 部署的位置 ≠ 風險位置，我們配置錯誤。
- (3) 誰負責在 **production** 出問題前抓到？——如果沒人 own，我們的命運是運氣。

這三個問題之外，所有 AI 策略討論都是次要的。」

本章小結

問	量化形式	立即 action
Q1：verification coverage	6 個層次 ≥ 90%	測量 + 設 CI gate
Q2：ratchet bite hardest	Incident / MTTR / 不敢動清單交集	部署測試 agent 到該區
Q3：who owns	Hybrid model + Quality team with power	Hire / appoint Q-arch

接下來

 [Chapter 05: 測試類型矩陣](#)

[← 回 README](#) · [← Ch 04](#)

第 05 章：測試類型矩陣

核心句：90% coverage 不是用一種測試達成的，是 6 種測試類型有機組合。每種測試抓不同層次的問題，搞混它們就等於建假防線。

6 種主要測試類型

類型	抓什麼	速度	寫的成本	AI 寫的能力
Unit	函式邏輯 bug	ms	低	★★★★★
Integration	模組間 contract 不一致	100ms	中	★★★★
E2E	完整用戶流程	分鐘	高	★★★
Property-based	邊界 case 與不變性	秒	中	★★★★
Mutation	假測試（測試品質保證）	分鐘	自動	n/a（infra）
Fuzz / Snapshot	未預期 input、UI 變動	秒	中	★★★★

Type 1：Unit Tests

定義：測單一函式 / class / 純邏輯，無 IO、無 DB、無 network。

典型例子：

```
def test_calculate_discount():
    assert calculate_discount(price=100, code='SAVE10') == 90
    assert calculate_discount(price=100, code=None) == 100
    assert calculate_discount(price=100, code='INVALID') == 100
```

何時用：

- 純函式（business logic、計算、轉換）
- Data structure 操作
- Validation 邏輯

何時不用：

- 牽涉外部依賴（DB / API）→ 用 Integration
- 跨模組互動 → 用 Integration
- UI 行為 → 用 E2E

AI 寫 unit test 的特殊優勢：

你給 AI	AI 給你
函式 signature + 1-2 個 example	完整 test suite（normal / edge / error）
函式實作	從實作 reverse engineer 出測試（注意：要 sanity check）

Bug fix PR

對應的 regression test

Prompt 範本：

Write comprehensive unit tests for this function.

Include:

- Normal cases (typical input)
- Edge cases (boundary values, empty, very large, very small)
- Error cases (invalid input, exceptions expected)
- At least 5 distinct test cases

Use pytest. Each test should have a descriptive name.

[paste function code]

Type 2 : Integration Tests

定義：測多個模組互動、有 DB / 有 API call、但仍在單一 process 內。

典型例子：

```
def test_user_registration_flow(db, mailer):
    response = client.post('/register', json={
        'email': 'foo@bar.com',
        'password': 'secret'
    })
    assert response.status_code == 201
    # DB 真寫入
    user = db.query(User).filter_by(email='foo@bar.com').first()
    assert user is not None
    # Email 真寄出
    assert mailer.last_sent.to == 'foo@bar.com'
```

何時用：

- API endpoint 行為
- DB schema + ORM
- 多 service 互動
- Auth / permission flow

何時不用：

- 純邏輯 → Unit
- 跨 service 跨機器 → E2E
- 第三方 API 真實互動 → 用 contract test，不是 integration

重要原則：整合測試打真 DB (test container)，不要全 mock。

過去 mock 流行是因為設置真 DB 慢。現在用 Docker / testcontainers，一切都是 throwaway。一條來自 12-factor 維護人的觀察：

「Mock 通過了，但 prod migration 失敗」——這就是 mock-everywhere 的代價。

Type 3 : E2E Tests

定義：模擬真實用戶從外部操作整個系統。瀏覽器 / API client → 多 service → DB。

典型例子（用 Playwright）：

```
test('user can complete checkout', async ({ page }) => {
  await page.goto('/');
  await page.click('text=Add to cart');
  await page.click('text=Checkout');
  await page.fill('input[name=email]', 'test@example.com');
  await page.fill('input[name=card]', '4242424242424242');
  await page.click('text=Pay $99');
  await expect(page.locator('text=Thank you')).toBeVisible();
});
```

何時用：

- 關鍵用戶流程（signup / checkout / cancellation）
- Cross-service 流程
- 看得到 / 摸得到的 UI 行為

何時不用：

- Edge case 細節 → Unit
- 模組互動 → Integration
- 慢、貴、不穩定，不該當主力

E2E 的數量原則：

5-15 個 critical journey 涵蓋核心商業流程。不要超過 **30 個**——E2E 跑太慢，flaky 率高，維護成本超過邊際效益。

Type 4 : Property-based Tests

定義：不是測「對某個 input X，輸出是 Y」，而是測「對所有 input，某個 property 永遠成立」。

典型例子（用 Hypothesis）：

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers()))
def test_sort_idempotent(lst):
    # property: sort 兩次跟 sort 一次結果一樣
    assert sorted(sorted(lst)) == sorted(lst)

@given(st.lists(st.integers()))
```

```
def test_sort_preserves_length(lst):  
    # property: sort 後長度不變  
    assert len(sorted(lst)) == len(lst)
```

Property-based test 工具會自動生成數百個 input 試圖打破 property。

何時用：

- 數學性質明確的函式 (sort、merge、compress)
- Idempotency (呼叫多次效果一樣)
- Invariant (系統不變性)
- Round-trip (encode + decode = identity)

何時不用：

- 沒有明確 mathematical property 的 UI / business logic
- 只關心特定 input → Unit

AI 寫 property-based test 的能力：★★★★。Prompt：

For this function, identify 3-5 mathematical/logical properties that should always hold. Then write Hypothesis (Python) / fast-check (JS) tests for each.

[paste function]

Type 5 : Mutation Testing

定義：測試你的測試。工具會故意改你的 code (變 + 成 -、變 < 成 <=) 然後跑你的測試。如果測試沒抓到，你的測試是假的。

工具：

- Python：mutmut, mutpy
- JS：Stryker
- Java：PIT
- Go：go-mutesting

典型輸出：

```
Total mutants: 234  
Killed (caught by tests): 187  
Survived (NOT caught): 47 ← 假測試!  
Mutation score: 80%
```

Mutation score 怎麼解讀：

分數	評估
< 50%	測試大部分是假的
50-70%	一半測試有效

分數 評估

70-85% 中位數

85-95% 良好

> 95% 卓越

Garry 的 90% coverage 應該對應 mutation score $\geq$ 75%。 否則就是 gaming。

何時用：

- Critical path 一定要跑
- Quarterly 全 codebase 跑
- PR 級別跑 diff mutation

何時不用：

- Mutation testing 很慢 ($\times 10$ test runtime)，不適合 every-commit
-

Type 6 : Fuzz Testing & Snapshot Testing

Fuzz

定義：用隨機 / structured-random input 砸你的函式，看會不會 crash / hang。

典型例子：

```
@hypothesis.given(st.binary())
def test_parse_doesnt_crash(data):
    try:
        my_parser.parse(data)
    except (ValueError, ParseError):
        pass # expected errors are OK
    # 但 segfault / hang / unhandled exception 就 fail
```

何時用：

- Parser、deserialization、user input handling
- 安全敏感 code

AI 寫 fuzz test：★★★★。Prompt:

Write fuzz tests for this parser.
Use Hypothesis to generate random bytes/strings.
Verify it never crashes (only raises expected exceptions).

[paste parser]

Snapshot

定義：把 function output 第一次跑的結果存起來 (snapshot)，之後每次跑跟 snapshot 比，不同就 fail。

何時用：

- UI 渲染 (React component → HTML snapshot)
- 大型 JSON / YAML 輸出
- 報告生成

陷阱：snapshot 容易「**meaningless update**」——壞掉直接 `---update-snapshots` 一鍵更新，等於沒測。人類 **review** 每次 **snapshot** 更新才有意義。

比例分配建議

90% coverage 的具體組合（典型 web service）：

70% Unit tests	← 速度快、寫得多、AI 友善
20% Integration tests	← 抓 contract 問題
5% E2E tests	← 5-15 個 critical journey
3% Property-based tests	← 關鍵函式
1% Fuzz / Snapshot	← 特殊場景

99%

外加：

- Mutation score：critical path 100%、aggregate $\geq 75\%$
 - Performance test：critical path 有 latency SLA 檢核
-

反例：常見比例錯誤

反例 1：100% E2E

「我們只寫 E2E，因為 unit test 太細」

問題：- E2E 跑超慢（30 分鐘 CI），開發者開始 skip - Flaky 高，造成 CI noise，慢慢沒人信任 - Bug 出現時找不到根因（E2E fail 通常只說「某處壞了」）

反例 2：100% Unit

「我們只寫 unit test，因為它快」

問題：- 模組間 contract 不一致時抓不到 - DB migration 不對抓不到 - API 改 schema 抓不到

反例 3：100% Mock

「所有測試都 mock 外部依賴，因為慢」

問題： - Mock 與 real 行為 drift（mock 永遠成功、real 偶爾失敗） - Refactor 時 mock 沒同步 → 假成功 - Mock pass 但 prod fail（最致命）

測試類型 vs ratchet 區域對照

回到 [Ch 04 Q2](#)：你應該在 ratchet bite hardest 的區域加最多測試。不同 **ratchet** 區用不同測試類型：

Ratchet 區	主要測試類型
業務邏輯	Unit + Property-based
Data schema	Integration + Migration test
整合層	Integration + Contract test
配置 / feature flag	E2E + Snapshot
無聲失敗	Mutation + Fuzz

對症下藥。

本章小結

類型	用途	比例
Unit	函式邏輯	70%
Integration	模組互動 / DB / API	20%
E2E	關鍵用戶流程	5%（5-15 個）
Property	數學性質	3%
Fuzz / Snapshot	特殊	1%
Mutation	測試品質保證	跑在上述之上

90% line coverage 加上 mutation score $\geq 75\%$ = 真 90%。

接下來

 [Chapter 06: AI 當測試作者 — 具體 SOP](#)

[← 回 README](#) · [← Ch 05](#)

第 06 章：AI 當測試作者 — 具體 SOP

核心句：「用 AI 寫測試」聽起來簡單，做不好就是「假覆蓋率」。本章給你 6 個工作流，把 AI 從「生產更多假測試的工具」變成「真把覆蓋率拉到 90% 且品質可信」的工具。

AI 寫測試的能力分布

不是所有測試類型 AI 都一樣強：

測試類型	AI 能力	為何
Unit test for pure function	★★★★★	LLM 強項：規律明確
Integration test	★★★★	需要 context (fixtures、env)
E2E test (Playwright)	★★★	需要 UI knowledge + selector 穩定性
Property-based test	★★★★	LLM 知道常見 properties
Mutation test	n/a	是 infra 不是寫測試
Fuzz test	★★★★	生成 strategies 容易
Snapshot test	★★★★★	純機械
Regression test from bug	★★★★★	LLM 很強，「重現再防」邏輯清楚

工作流 1：補測試到 90%

情境：你的 codebase 目前 65% coverage，要拉到 90%。

SOP

- Step 1: 跑 coverage 找未覆蓋的 file/line
- Step 2: 用 LLM 為每個 file 生成測試 (批次)
- Step 3: 跑測試，看 pass 率
- Step 4: 失敗的測試人類 review (通常是 LLM 誤解 implementation)
- Step 5: 跑 mutation testing 抓假測試
- Step 6: 假測試重生 (給 LLM 看為何被 mutate 殺死)

具體腳本

```
#!/bin/bash
# generate-tests.sh

# Step 1: 找未覆蓋檔案
pytest --cov=src --cov-report=json:.coverage.json
UNCOVERED_FILES=$(jq -r '.files | to_entries[] | select(.value.summary.percent_covered < 90) | .key' .coverage.json)

# Step 2: 為每個檔案生成測試
for file in $UNCOVERED_FILES; do
  echo "Generating tests for $file..."

  # 把檔案內容 + existing tests 餵給 LLM
  EXISTING_TESTS=$(find tests/ -name "*${basename $file .py}*" -exec cat {} \;)
```

```
claude code "  
Write comprehensive pytest tests for this file to reach 90% coverage.  
  
EXISTING TESTS (don't duplicate):  
$EXISTING_TESTS  
  
SOURCE FILE:  
$(cat $file)  
  
REQUIREMENTS:  
- Use pytest  
- Test all branches (not just happy path)  
- Use fixtures where appropriate  
- Each test has descriptive name  
- Output only the test code, no explanation  
" > tests/test_$(basename $file)  
done  
  
# Step 3: 跑測試  
pytest --cov=src --cov-fail-under=90
```

Common pitfall

❌ LLM 直接從 **implementation** 推測試——這會「通過但無意義」（測試 = implementation 的鏡像）

✅ LLM 從 **docstring / spec / behavior** 推測試——這才能抓到 implementation bug

加 prompt :

IMPORTANT: Write tests based on the docstring and intended behavior,
NOT by mirroring the implementation. If implementation has bugs,
the test should catch them.

工作流 2：每個 bug fix 都寫 regression test

情境：production 出 bug → fix → 防止再發生。

SOP

Step 1: 寫一個 failing test 重現 bug (在 fix 之前)
Step 2: 寫 fix
Step 3: 確認 test 通過
Step 4: 把 test commit 進 regression suite

AI 加速

修 bug 時 prompt :

Here's a bug report:
[paste bug description / stack trace / Sentry issue]

Here's the relevant code:
[paste code]

Tasks:

1. Write a failing pytest test that reproduces this bug (before fixing)
2. Then write the minimal fix
3. Then explain why the test would have caught this earlier

Output:

- test_<descriptive>.py (failing test)
- fix in src/<file>.py
- 1-paragraph postmortem

結果：每修一個 bug，多一個 **regression test**。Coverage 跟 quality 一起拉高。

工作流 3：PR 時自動生成 diff coverage 測試

情境：保證每個 PR 不降低 coverage。

CI 設定

```
# .github/workflows/test.yml
name: Test
on: pull_request

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # 需要 git history

      - run: pip install -r requirements.txt

      - name: Run tests with coverage
        run: pytest --cov=src --cov-report=xml

      - name: Diff coverage check
        run: |
          diff-cover coverage.xml --compare-branch=origin/main \
            --fail-under=90 # diff 必須 90% 覆蓋

      - name: Total coverage gate
        run: |
          coverage report --fail-under=85 # 整體不能低於 85 (現在)
```

如果 PR diff coverage 不足

提示 PR 作者用 AI 補：

```
# .github/hooks/post-commit (optional)
DIFF_COV=$(diff-cover coverage.xml --compare-branch=origin/main | grep "Coverage:" | awk
  '{print $2}')
if [ $(echo "$DIFF_COV < 90" | bc) -eq 1 ]; then
  echo "⚠️ Diff coverage $DIFF_COV < 90%"
```

```
echo "Run: claude code 'generate tests for new uncovered lines'"
fi
```

workflow 4：用 AI 抓「假測試」

情境：90% coverage 達到了，但你懷疑很多測試是「**assert True** 等級的假測試」。

三層防線

防線 1：AI code-review 測試

每個 PR 的測試檔案，跑 AI 評審：

```
claude code-review --focus=tests "
Review these test files for quality issues:
- assert True / trivial assertions
- Tests that don't actually verify behavior
- Mocked everything (mock-mock-pass)
- Tests that mirror implementation (no value)
- Missing edge cases

Files: [list]
"
```

防線 2：Mutation testing 跑批次

```
# 每週日跑全 codebase mutation
mutmut run --paths-to-mutate src/
mutmut results > mutation-report.txt

# 警報:mutation score < 75%
SCORE=$(jq '.score' mutation-results.json)
if [ $(echo "$SCORE < 75" | bc) -eq 1 ]; then
    slack-notify "#engineering" "⚠️ Mutation score dropped to $SCORE%"
fi
```

防線 3：人類 review 隨機抽樣

每週 senior engineer 隨機抽 10 個測試，跑這 prompt：

```
For each test below, evaluate:
1. Does it actually verify behavior? (Y/N)
2. Would it catch a real bug in the function? (Y/N)
3. If implementation had off-by-one error, would this test catch it? (Y/N)

[paste 10 random tests]
```

如果 < 7/10 是真測試，你的測試套基礎有問題——回去用 workflow 1 重做。

workflow 5：從 spec / PRD 寫 E2E

情境：產品經理寫了一份 PRD，工程師要實作 + 測試。

反 patternsully：先寫 code 後補 E2E

問題：- 工程師寫完 code，testing 是「驗證 code 做了我寫的東西」（同義反覆）- 沒抓到「code 沒做 PRD 寫的東西」

Pattern：PRD → E2E → Code → Iteration

Step 1: PRD 完成
Step 2: 用 AI 從 PRD 生成 E2E test (用 Playwright / Cypress)
Step 3: E2E test 跑，全部 fail (because no code yet)
Step 4: 工程師寫 code 讓 E2E pass
Step 5: PRD 改動，E2E 同步改動

AI Prompt

Here's the PRD:
[paste PRD]

Generate Playwright E2E tests covering:
1. Happy path (the primary user journey)
2. Error scenarios (what user sees when X fails)
3. Edge cases (empty state, max state, etc.)

Tests should be:
- Independent (no shared state)
- Deterministic (no flaky timing)
- Self-cleanup (set up + tear down)

Output: Playwright TypeScript code, ready to run.

這就是 **outside-in TDD** 在 AI 時代的具體形式。

Workflow 6：AI 生成 Property-based test

情境：你有個 critical 函式，想確保所有 input 行為正確，不只是手寫的 case。

Prompt

Function:
[paste function with docstring]

Tasks:
1. Identify 3-5 mathematical/logical PROPERTIES this function should always satisfy.
2. For each property, write a Hypothesis test.
3. Explain in 1 sentence per property why it must hold.

Format:
- # Property 1: <description>
 # Why: <reasoning>
 @given(...)
 def test_property_1(...): ...

- # Property 2: ...

Example properties to consider:

- Idempotency: $f(f(x)) == f(x)$
 - Commutativity: $f(a, b) == f(b, a)$
 - Identity element: $f(x, \text{identity}) == x$
 - Inverse: $g(f(x)) == x$
 - Length preservation: $\text{len}(f(xs)) == \text{len}(xs)$
 - Monotonicity: $a < b \rightarrow f(a) \leq f(b)$
 - Determinism: $f(x) == f(x)$ given same input
-

整合：AI 測試生成的 daily SOP

把這些 workflows 組合成日常：

觸發	動作	工具
PR submit	AI 生成 diff coverage 測試	CI hook + LLM
Bug report	AI 寫 regression test	工作流 2
New PRD	AI 生成 E2E skeleton	工作流 5
Refactor critical function	AI 加 property-based test	工作流 6
每週	Mutation testing 跑批次	mutmut + cron
每月	Senior engineer 隨機抽樣 review	工作流 4 防線 3
每季	全 codebase verification audit	用 Ch 04 檢核表

AI 寫測試的 5 個鐵則

1. AI 寫測試 $\neq$ 不需要 review：每個 AI 生成的測試都要人類過 1 眼
 2. Mutation testing 是必要的「測試品質檢核員」：沒有它，AI 寫的假測試會大量滲入
 3. Prompt 要求「從 spec 而非 implementation」：避免測試與實作同義反覆
 4. 小批次生成，不要整 codebase 一次餵：LLM context 越短品質越穩
 5. 每個 mutation killed by your test = 你的測試品質指標：把這個數字當成 KPI
-

反例集錦

反例 1：「我用 AI 一鍵生成所有測試，coverage 90%」

問題：mutation score 可能 $< 50\%$ 。一堆假測試。

解：mutation testing 必跑，作為「真假 90%」的二階檢核。

反例 2：「AI 生成的測試我都接受，因為它們 pass」

問題：通過 $\neq$ 有意義。可能是 assert True 或測 implementation 的鏡像。

解：每個測試 review 「這在抓什麼 **bug** ?」 答不出來就丟。

反例 3：「我用 AI mock 所有外部依賴」

問題：mock 永遠成功，prod 偶爾失敗。

解：用 testcontainers 跑真 DB / 真 service，AI 只 mock 慢的、不穩的、不重要的。

本章小結

workflow	觸發	結果
1. 補測試到 90%	一次性 migration	65% → 90% in 3 週
2. Bug fix → regression	每次 bug	防止 regression
3. PR diff coverage	每個 PR	不降低 coverage
4. 抓假測試	每週 / 隨機	保品質
5. PRD → E2E	新 feature	Outside-in 開發
6. Property-based	Refactor	數學保證

接下來

 [Chapter 07: 反模式 — 假測試陷阱](#)

[← 回 README](#) · [← Ch 06](#)

第 07 章：反模式 — 假測試陷阱

核心句：80% 覆蓋率比 60% 危險——因為它讓你以為有防線。本章列 10 個 anti-patterns，幫你識別「看起來像測試但實際上是裝飾」的陷阱。

反模式 #1：assert True

✗ 錯：

```
def test_calculate_price():
    result = calculate_price(items=[item1, item2])
    assert True # 啊? 沒測什麼
```

為什麼壞：執行了 code，但沒驗證任何行為。Coverage 漂亮，零價值。

識別信號：- assert True - assert 1 == 1 - assert response is not None (只測沒 crash)

✓ 正解：

```
def test_calculate_price():
    result = calculate_price(items=[
        Item(price=100), Item(price=200)
    ])
    assert result == 300
```

反模式 #2：測試是 Implementation 的鏡像

✗ 錯：

```
# implementation
def calculate_tax(price):
    return price * 0.08

# test
def test_calculate_tax():
    assert calculate_tax(100) == 100 * 0.08 # ← 同一個 0.08
```

為什麼壞：如果 0.08 應該是 0.10（程式 bug），測試也是 0.08，永遠抓不到。測試跟著程式一起錯。

✓ 正解：用 explicit expected value：

```
def test_calculate_tax():
    # 期望：8% 稅率 (從 spec)
    assert calculate_tax(100) == 8.0
    assert calculate_tax(50) == 4.0
    assert calculate_tax(1000) == 80.0
```

如果 spec 改 tax rate，你只改 test 對應的 expected，不會自動跟程式錯。

反模式 #3：Mock 整個世界

✗ 錯：

```
def test_user_signup(mock):
    mock.patch('db.User.save')           # mock DB
    mock.patch('mailer.send')           # mock email
    mock.patch('redis.set')             # mock cache
    mock.patch('sentry.capture')         # mock error tracker
    mock.patch('analytics.track')       # mock analytics

    user_signup(email='foo@bar.com', password='x')

    db.User.save.assert_called_once()    # 只驗證「**call 過了**」
```

為什麼壞：你測的不是「user signup 正確」，是「這些 mock 被呼叫了」。實際 DB schema 改了、real mailer 改了，這測試還是 pass。Mock pass，prod fail。

✓ 正解：用 testcontainers 跑真 DB + 真 service：

```
def test_user_signup(db_container): # 真 PostgreSQL container
    user_signup(email='foo@bar.com', password='x')

    # 從真 DB 查
    user = db_container.session.query(User).filter_by(
        email='foo@bar.com'
    ).first()
    assert user is not None
    assert user.password_hash != 'x' # 確實被 hash 了
```

Mock 是工具不是教義。Mock 慢的、外部的、隨機的；不要 mock 你的 schema、你的核心邏輯。

反模式 #4：超大 Test，沒有 isolation

❌ 錯：

```
def test_full_workflow():
    # 30 行 setup
    user = create_user(...)
    org = create_org(user)
    project = create_project(org)
    issue = create_issue(project)

    # 50 行 action
    add_comment(issue, ...)
    assign_user(issue, ...)
    update_status(issue, ...)

    # 20 行 assert
    assert issue.comments[0].text == ...
    assert issue.assignee == ...
    assert issue.status == ...
```

為什麼壞：- 任一行壞，整個 test fail - 不知道是哪個 action 出問題 - 失敗時 debug 1 小時起跳 - LLM 也理解不了「這個 test 到底在測什麼」

✅ 正解：每個 test 一個 action：

```
def test_add_comment(issue_factory):
    issue = issue_factory()
    add_comment(issue, 'hello')
    assert issue.comments[0].text == 'hello'

def test_assign_user(issue_factory, user_factory):
    issue = issue_factory()
    user = user_factory()
    assign_user(issue, user)
    assert issue.assignee == user

# ... 各自獨立
```

Test 一行 setup、一行 action、一行 assert 是聖經。

反模式 #5 : Skipping Tests 卻不刪

❌ 錯：

```
@pytest.mark.skip(reason="flaky, fix later")
def test_complex_scenario():
    ...

@pytest.mark.xfail
def test_another_thing():
    ...
```

為什麼壞：- Skip 的測試從不被刪除也不被修，永遠在 codebase 裡裝樣子 - Coverage 報告仍可能算入（看工具） - 新工程師看到 skip 不知道是不是還有意義 - 跟 `assert True` 一樣等於沒測

✅ 正解：- 一週內修不好 → 刪掉 - 真的需要保留 → 加 **expiry** 與 **owner**：

```
@pytest.mark.skip(
    reason="DB locked race condition - tracked in JIRA-123",
    # SKIP_EXPIRES: 2026-06-01
    # OWNER: jane@example.com
)
def test_complex_scenario():
    ...
```

跟 CI 整合：超過 expiry 還 skip → CI fail，強迫處理。

反模式 #6：用 try/except 包整個 Test Body

❌ 錯：

```
def test_thing():
    try:
        result = my_function()
        assert result == expected
    except Exception:
        pass # 包住所有錯誤
```

為什麼壞：你的 test 永遠 pass，不管 `my_function` 是不是真壞了。

✅ 正解：只 catch 預期的 exception，並斷言它確實發生：

```
def test_thing_raises_on_invalid():
    with pytest.raises(ValueError, match="invalid input"):
        my_function(invalid_arg)
```

反模式 #7：依賴 Test 執行順序

❌ 錯：

```
def test_1_create_user():
    global user_id
    user_id = create_user('foo')
    assert user_id

def test_2_login_user():
    response = login(user_id) # ← 依賴 test_1 跑過
    assert response.status == 200
```

為什麼壞：- 換順序就壞 - 平行跑就壞 - 沒辦法 isolate debug 單一 test

✅ 正解：每個 test 自己 setup（用 fixture 共享）：

```
@pytest.fixture
def user():
    return create_user('foo')

def test_login(user):
    response = login(user.id)
    assert response.status == 200
```

反模式 #8：Flaky test 容忍症

❌ 錯：CI 有 5% test flaky。團隊「習以為常」，看到 fail 就 retry，過了就 merge。

為什麼壞：- 真 bug 跟 flaky 混在一起，分不清 - 信任度崩盤——CI fail 大家先 retry - 慢慢更多 test 變 flaky，但沒人警告 - 直到某天 prod 壞了，回查發現原來那個 flaky test 真的在抓 bug

✅ 正解：Zero tolerance

```
# 每個 PR 跑 3 次，3 次都 pass 才算 pass
pytest --reruns=0 --maxfail=1 # 嚴格模式

# 偵測 flaky
pytest --count=10 # 跑 10 次，有任何不一致就 fail
```

Flaky test 出現：1. 立即 isolate（mark + 報警，不直接 **disable**）2. 24 小時內 root cause 3. 修不好就刪

反模式 #9：以 LoC 為 Test KPI

❌ 錯：「這個 sprint 我們寫了 500 行 test」當成功標記。

為什麼壞：- 鼓勵寫多而非好 - AI 一秒鐘可以寫 5000 行 garbage test - LoC 跟「抓到的 bug 數」沒關係

✅ 正解：用這些 KPI：- **Mutation score**（測試品質）- **Regression test count**（每修一個 bug 加一個）- **Critical path coverage**（高風險區 100%）- **CI confidence**（綠燈才能上 prod 的信任度）

LoC 不要當 KPI。

反模式 #10：「我們有 manual QA 所以不需要這麼多自動測試」

✗ 錯：「Manual QA team 跑 regression suite，自動測試 60% 就夠」

為什麼壞：- Manual QA 一週只能跑 1-2 個完整 regression cycle - AI agent 一週可以塞 50 個 PR - 數學上 manual QA 永遠追不上

✓ 正解：把 manual QA 角色升級：

Manual QA 從做	升級成做
跑 regression（重複勞動）	設計新測試（高槓桿）
手動驗證 happy path	寫 exploratory testing tool
寫 bug report	寫 root cause analysis + 防 regression test
Test 執行	Test architecture

Manual QA 沒有消失，是被「升維」——做機器做不了的事。

反模式對照速查

#	反模式	識別信號	解法
1	assert True	沒有 assertion	用 explicit value
2	Implementation 鏡像	Test 跟 code 用同 constant	用 spec-derived value
3	Mock 整個世界	一堆 patch	用 testcontainers
4	巨型 test	100+ 行	拆成單一 action
5	不刪的 skip	@pytest.mark.skip 沒 expiry	加 expiry + owner
6	try/except 包 test	bare except	用 pytest.raises
7	依賴順序	global state	用 fixture
8	Flaky 容忍	retry 文化	Zero tolerance
9	LoC 當 KPI	周報講行數	用 mutation score
10	Manual QA 替代	60% 覆蓋率自滿	升級 QA 角色

自我檢查腳本

把這個跑你 codebase 一遍：

```
#!/bin/bash
# detect-anti-patterns.sh

echo "=== 反模式 #1: assert True / 平凡 assertion ==="
grep -rn 'assert True\|assert 1 == 1\|assert None is None' tests/ || echo "✓ none"
```

```

echo "=== 反模式 #5: skip 沒 expiry ==="
grep -rn 'pytest.mark.skip\|@skip' tests/ | grep -v 'SKIP_EXPIRES' || echo "✓ all good"

echo "=== 反模式 #6: bare except ==="
grep -rn 'except:\|except Exception:\s*pass' tests/ || echo "✓ none"

echo "=== 反模式 #7: global state in tests ==="
grep -rn '^global ' tests/ || echo "✓ none"

echo "=== Test 行數分布 (找太大的 test) ==="
awk '/^def test_{n=NR; name=$0} /^def [^t]{if(n)print NR-n, name; n=0}' tests/*.py | sort
-n -r | head -10

echo "=== 跑 mutation testing (基礎品質指標) ==="
mutmut run 2>&1 | tail -3

```

本章小結

反模式	危害程度	修復難度
#1 assert True	高	低
#2 Impl 鏡像	致命 (看不到 bug)	中
#3 Mock 全世界	致命 (mock pass prod fail)	中
#4 巨型 test	中 (debug 痛苦)	低
#5 不刪 skip	中 (累積技術債)	低
#6 try/except 包 test	高	低
#7 順序依賴	中	低
#8 Flaky 容忍	致命 (信任度崩盤)	高
#9 LoC KPI	中	低
#10 Manual QA 替代	中	中

修這 10 個之前，講 90% 覆蓋率都是空話。

接下來

 [Chapter 08: 12 週實施路線圖](#)

[← 回 README](#) · [← Ch 07](#)

第 08 章：12 週實施路線圖 — 從 65% 拉到 90%

核心句：把 Garry Tan 的 90% 目標拆成 12 週可執行計畫。每週有 deliverable、有 owner、有檢核點。把這份當作下個 quarter 的工程 OKR。

前提假設

條件	數值
目前 line coverage	65%
Mutation score	未測量 / < 50%
工程團隊	10-30 人
主要技術 stack	Python/TypeScript/Go (任一)
AI 工具預算	\$1000-3000/月
目標	第 12 週末 ≥ 90% line, ≥ 75% mutation

路線圖總覽

Week 1-2: 測量 + 立 baseline
Week 3-4: 攻擊 Critical Path (最高優先)
Week 5-6: Business Logic 補測試
Week 7-8: Integration & E2E
Week 9-10: Mutation testing + 假測試清除
Week 11: CI hardening + diff coverage gate
Week 12: 全面 audit + 90% 確認

Phase 1：測量 + 立 baseline (Week 1-2)

目標：知道現狀，定 KPI。

Week 1: 測量

- ☐ 跑 coverage report，記錄每個 **module** 的覆蓋率
- ☐ 跑 mutation testing 在 critical path (至少 3 個重要 module)
- ☐ 列出 [Ch 04](#) 9 項 verification 維度的當前分數
- ☐ 列出過去 12 個月 P0/P1 incident 對應的系統
- ☐ 問所有 senior：「不敢動」清單

Week 2: 立 baseline

- ☐ 寫一份 1 頁 report 給 leadership：「**Coverage 現狀 + 12 週計畫**」
- ☐ CI 加 --fail-under=65 (從現狀) 並 commit
- ☐ 指派 Quality Lead (有阻擋 PR 權力)
- ☐ 訂 90% 目標、訂 mutation score ≥ 75% 目標
- ☐ 公開 dashboard (顯示 coverage、mutation、reg test count)

Phase 1 deliverable

1. Coverage baseline report

2. CI 強制執行 --fail-under=65
 3. Quality Lead appointed
 4. Public dashboard
-

Phase 2 : 攻擊 Critical Path (Week 3-4)

目標：高風險系統 100% 覆蓋。

從 [Ch 04 Q2](#) 找出 ratchet bite hardest 的 3 個系統，這兩週只攻它們。

Week 3-4: Critical Path 100%

每個 critical 系統：- [] 列出所有 entry function (API endpoint、business logic 入口) - [] 用 AI 寫 unit tests ([Ch 06 工作流 1](#)) - [] 人類 review 每個 test - [] 跑 mutation testing，補假測試 ([工作流 4](#)) - [] 確認該系統覆蓋率 $\geq 95\%$, mutation $\geq 80\%$

Phase 2 deliverable

- 3 個 critical 系統覆蓋率 100%
 - Mutation score $\geq 80\%$
 - 整體覆蓋率從 65% $\rightarrow$ 70%
-

Phase 3 : Business Logic 補測試 (Week 5-6)

目標：非 critical 但常改的 business logic 拉到 90%。

Week 5: Identify

- ☐ 列出 git log --since="1 year ago" --name-only 改最多次的 file
- ☐ 取 top 50% (這些是常改的、需要 regression 保護)
- ☐ 排除已在 critical path 的

Week 6: AI Mass Generation

- ☐ 用 [工作流 1](#) 批次生成
- ☐ 每天 50-100 個 test
- ☐ 人類 review SLA 24 小時
- ☐ Reject 比例不能 $> 30\%$ (太高代表 prompt 還沒調好)

Phase 3 deliverable

- 高頻 file 覆蓋率 $\geq 90\%$
 - 整體 65% $\rightarrow$ 78%
-

Phase 4 : Integration & E2E (Week 7-8)

目標：模組間 contract + 關鍵用戶流程。

Week 7: Integration

- ☐ 列出所有 API endpoint
- ☐ 每個 endpoint 寫 integration test (真 DB, 不是 mock)
- ☐ 用 testcontainers / docker-compose

Week 8: E2E

- ☐ 列出 5-15 個 critical journey
- ☐ 用 Playwright / Cypress 寫 E2E
- ☐ 用 [workflow 5](#) 從 PRD 生成
- ☐ CI 跑 E2E (限制: parallel + retries=0)

Phase 4 deliverable

- API endpoint 100% integration covered
 - 5-15 個 critical E2E journey 跑得穩
 - 整體 78% → 85%
-

Phase 5 : Mutation + 假測試清除 (Week 9-10)

目標：mutation score $\geq 75\%$ ，徹底清除假測試。

Week 9: Mutation 全面跑

- ☐ 跑 mutation testing 在整個 codebase
- ☐ 列出 surviving mutants (沒被測試殺死的)
- ☐ 把 mutation result 餵 LLM，請它改善 existing tests

Week 10: 假測試手動 audit

- ☐ Senior engineer 隨機抽 100 個 test
- ☐ 跑 [Ch 07](#) 的反模式檢查腳本
- ☐ 刪掉 / 修復假測試 (預期 10-20% 需要動)

Phase 5 deliverable

- Mutation score: 50% → 75%
 - Anti-pattern detection 全 pass
 - 整體 85% → 88%
-

Phase 6 : CI 加固 + Diff Coverage Gate (Week 11)

目標：守住 90%，不退步。

Week 11

- ☐ CI 加 diff coverage gate：每 PR 新行 $\geq 90\%$ 覆蓋
- ☐ CI 加 mutation gate：critical path 每 PR mutation $\geq 80\%$
- ☐ CI 加 anti-pattern 檢查 ([Ch 07 腳本](#))
- ☐ 升 `--fail-under` 從 65% $\rightarrow$ 88%
- ☐ 公告：超過 88% 的 module 才能 merge，例外要 escalate

Phase 6 deliverable

- CI 鎖住「不能退步」
 - 全組織意識到「測試是 first-class」
-

Phase 7 : 最終 audit + 90% confirmation (Week 12)

目標：證明 90% 達成且有意義。

Week 12

- ☐ 整體 coverage 跑一次最終 report
- ☐ Mutation score 跑一次最終 report
- ☐ [Ch 04](#) 9 維度重新打分 (目標： ≥ 80)
- ☐ 跟 Quality Lead 對 critical path：100% line, $\geq 80\%$ mutation
- ☐ CI `--fail-under=90`
- ☐ 寫 12 週 retrospective：什麼有效、什麼沒效、Q3 計畫
- ☐ 慶祝 + 給團隊 credit

Phase 7 deliverable

- 整體 $\geq 90\%$ line, $\geq 75\%$ mutation
 - Critical path 100% line, $\geq 80\%$ mutation
 - CI 強制 90%
-

週度 standup template

每週 Quality Lead 帶這個 standup：

Quality Weekly – Week N

Coverage

- 整體：__% (本週 +/- %)
- Critical path: __%
- Mutation: __%

****This week****

- Wins:
- Blockers:
- AI gen ratio (人類 review pass rate): __%

****Risks****

- Module X 覆蓋率仍 < 60%, owner: ____
- Y 個 flaky test 待處理

****Next week****

- 攻擊 module Z
- 完成 ____

****Help needed****

- ____

如果你預算 / 人力更少

縮短版（6 週）：

Week	行動
1	測量 + 列 critical 系統
2-3	AI 補 critical 系統測試
4	Mutation testing + 假測試清除
5	Integration + 1-2 個 critical E2E
6	CI gate + audit

目標調整：從 90% 改 80%（但 critical path 仍 100%）。

如果你預算 / 人力更多

加速版（8 週達成 + 後續強化）：

Week	額外項目
1-2	+ 全 codebase mutation testing 跑出 baseline
3-4	+ 平行 5 個 critical 系統
5-6	+ Performance test + chaos engineering
7-8	+ 全 codebase diff mutation in CI

達成 90% 之後做什麼

繼續維持是一回事，更高層次的工作：

- 1. **Verification observability**：在 production 直接 inject 故障測試（chaos engineering）
- 2. **Contract testing** 跨 service
- 3. **Performance regression** budget
- 4. **Security test** 整合（SAST / DAST 進 CI）
- 5. **AI 測試品質的 ML 評估**（更 advanced 的 mutation 變體）

但這些都建立在「已有 **90%** 真品質覆蓋率」之上。先做好 baseline，再玩進階。

預算估算

項目	12 週成本
Quality Lead（兼任 30% time）	內部資源
AI 工具（Claude / Cursor）	\$200-500/月 × 12 週 = \$600-1500
Mutation testing infra（CI 額外計算）	\$200-500/月
Testcontainers infra	通常已有
人類 review 時數（4 senior × 4 hrs/week × 12 週）	\$30K
總計	~\$35-45K

ROI 比較：一個 P0 incident 平均 \$50K-200K（service downtime + 工程師時間 + customer churn）。**12 週投資 < 一次大 incident 損失。**

本章小結

Phase	週次	目標
1. 測量 + baseline	1-2	知現狀
2. Critical path	3-4	高風險系統 100%
3. Business logic	5-6	常改 file 90%
4. Integration + E2E	7-8	API + journey
5. Mutation + 清假	9-10	品質保證
6. CI 加固	11	守 90%
7. 最終 audit	12	確認達成

12 週後：**90% line, 75% mutation, CI 強制**。組織完成 AI 時代的驗證能力升級。

接下來

第 09 章：跟 12-Factor Agents 的關聯

核心句：Garry Tan 的「90% test coverage」跟 Dex Horthy 的「12-factor agents」乍看談不同問題。實際上它們是「LLM 時代 **production-grade** 軟體」這枚硬幣的兩面——12-factor 是建造側、90% 是驗證側。沒有任一面，AI 應用都上不了 customer-facing production。

兩條觀念的關係

觀念	提出者	解決問題
12-factor agents	Dex Horthy	怎麼建造可控的 LLM 應用
90% test coverage	Garry Tan	怎麼驗證 AI 產出的軟體

12-factor 是「寫對」，**90%** 是「確認沒寫錯」。一個是 input quality、一個是 output quality。

為什麼這兩條必須同時存在

只有 12-factor 而沒有 90%：- Agent 架構漂亮、可控、可暫停 - 但你不知道它在 **production** 會不會壞——AI agent 寫的 deterministic 程式碼可能有 bug 你沒抓到 - 結果：可控的 agent 還是會出 incident

只有 90% 而沒有 12-factor：- Agent 寫的 code 都被測試覆蓋 - 但 **agent** 本身是黑盒，走火入魔時沒人能介入 - 結果：production agent 跑 5 分鐘就 spin out

兩條一起做，你才得到「**production-grade AI 應用**」。

對應映射

Garry Q1 (verification coverage) ↔ 12-factor Factor 9 (Compact Errors)

Garry Q1: What is our verification coverage on AI outputs?

Factor 9: 把錯誤壓進 context window，讓 LLM 自癒

兩條都在問「錯誤怎麼被抓到 + 修復」。

- **Factor 9** 處理「運行時錯誤」：tool call 失敗時，LLM 看到 stack trace 後嘗試自我修復
- **Garry Q1** 處理「部署前錯誤」：測試體系抓到 bug，在 PR 階段就 reject

兩個都是「錯誤捕獲」，只是在不同 stage。

Garry Q2 (ratchet bite hardest) ↔ 12-factor Factor 10 (Small Focused Agents)

Garry Q2: Where does the ratchet bite hardest?

Factor 10: Small, focused agents (3-10 step)

兩條都在處理「複雜度爆炸」。

- **Factor 10** 從結構上避免複雜度：把 agent 拆小，每個只處理一件事
- **Garry Q2** 從驗證上對抗複雜度：找出複雜度最大的區域，集中防線

結構限制 + 驗證強化，是對抗複雜度的兩支腳。

Garry Q3 (who owns) ↔ 12-factor Factor 7 (Contact Humans with Tools)

Garry Q3: Who owns catching regressions?

Factor 7: 用 tool call 聯絡人類（人類介入機制）

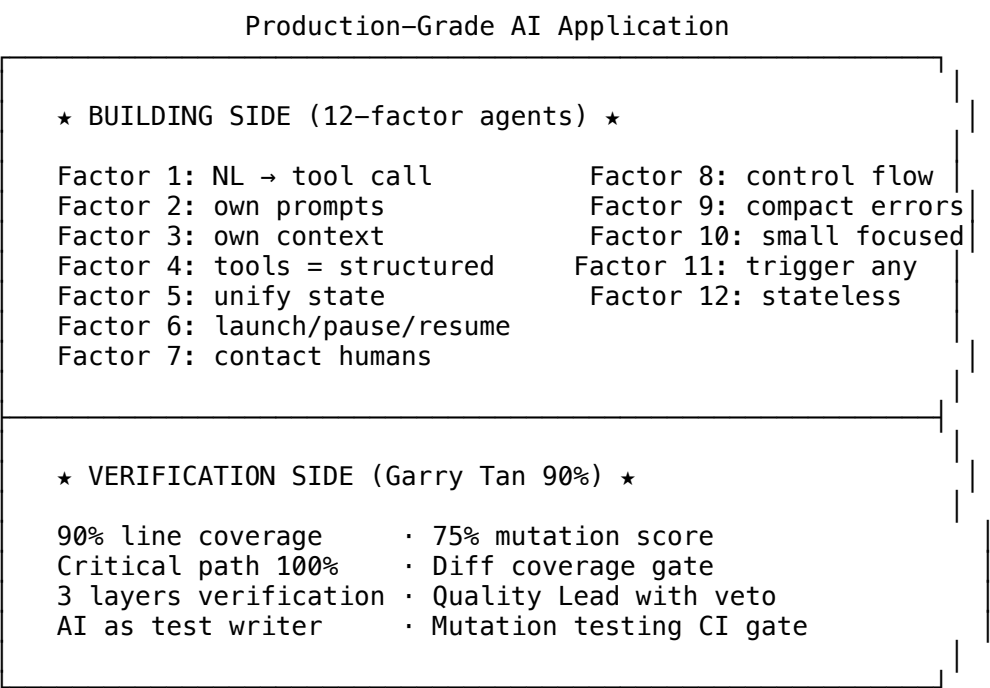
兩條都在處理「人類在 loop 的角色」。

- **Factor 7** 設計運行時人類介入：agent 跑到關鍵點主動找人類
- **Garry Q3** 設計組織人類介入：明確誰負責抓 regression

人類不是裝飾，是 **critical infrastructure**。

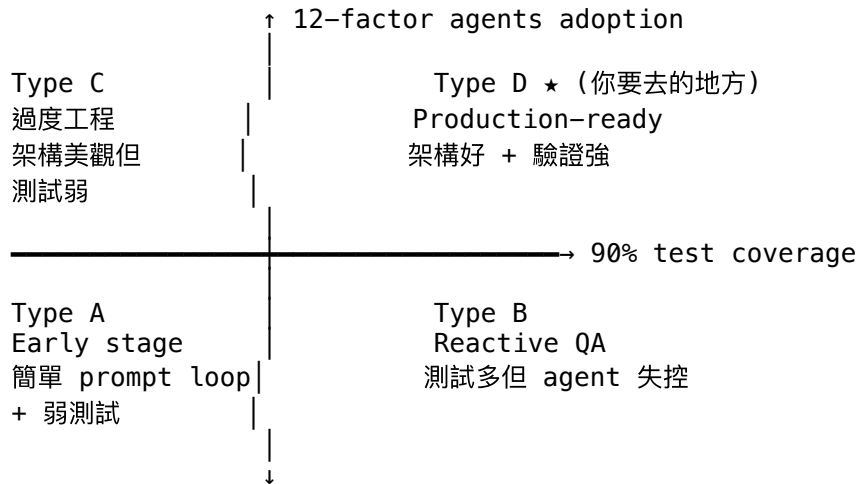
完整 production architecture

把兩條觀念合成完整架構：



工程組織的 4 種狀態

根據兩條原則的完成度，組織分 4 種：



多數 AI 新創 **stuck** 在 **Type A**（沒測試也沒架構） 部分大公司在 **Type C**（架構漂亮但沒測試） 少數成熟公司在 **Type B**（測試多但 **agent** 鬆散） 極少數在 **Type D**（兩者都做）

Garry + Dex 的方法論合起來，就是把你從任何起點推到 **Type D** 的 **playbook**。

整合範例：12-factor agent 的 verification 體系

把兩條觀念套到一個具體 deploybot 案例：

12-factor 設計

```
# 標準 12-factor agent 架構
class DeployAgent:
    def handle_next_step(self, thread: Thread):
        while True:
            next_step = self.llm.determine_next_step(thread_to_prompt(thread))

            if next_step.intent == 'deploy_backend_to_prod':
                # high-stakes: 需要人類審批
                self.request_human_approval(next_step)
                self.save_thread(thread)
                break
            elif next_step.intent == 'list_git_tags':
                tags = fetch_git_tags()
                thread.events.append({'type': 'list_git_tags_result', 'data': tags})
                continue
            # ... 其他 case
```

Garry 90% 驗證體系

對應的測試 suite :

```
# Unit tests
def test_handle_next_step_deploy_intent():
    """高 stakes deploy intent 必須 break loop 等人類"""
    agent = DeployAgent(mock_llm_returning('deploy_backend_to_prod'))
    thread = Thread()
    agent.handle_next_step(thread)
    assert mock_request_human_approval.called
    assert thread.is_saved

def test_handle_next_step_list_tags_intent():
    """List tags 同步執行不 break"""
    agent = DeployAgent(mock_llm_returning('list_git_tags'))
    thread = Thread()
    # 應該 continue loop 不 break
    ...

# Integration tests
def test_full_deploy_flow_with_real_db(db_container):
    """完整 deploy flow: trigger → LLM decisions → human approval → execute"""
    ...

# E2E tests
def test_slack_triggered_deploy_e2e(playwright):
    """從 Slack message 觸發到 deploy 完成的完整 journey"""
    ...

# Property-based tests
@given(thread_strategy)
def test_thread_serialization_round_trip(thread):
    """Property: thread serialize + deserialize = identity (Factor 5/12)"""
    serialized = json.dumps(thread.to_dict())
    restored = Thread.from_dict(json.loads(serialized))
    assert restored == thread

# Mutation testing
# 跑 mutmut, 確認 critical control flow 的 mutation 都被殺死
```

這套測試對應 **12-factor** 的每個 **factor** : - Factor 8 (control flow) → unit tests for switch cases - Factor 5 (state) → property-based serialization round-trip - Factor 7 (human) → integration test 確認 approval 路徑 - Factor 6 (pause/resume) → integration test 確認 save/load thread - Factor 11 (trigger anywhere) → E2E test 從 Slack 觸發

1+1 > 2 的綜效

場景	只有 12-factor	只有 90%	兩者皆有
Agent 走火入魔	Factor 10 防範 + Factor 9 自癒	測試抓不到 runtime 問題	★ 結構限制 + 運行時自癒 + 測試保底
Bug 進 production	架構好但仍可能漏	測試抓到	★ 雙重防線
Human-in-loop 失效	Factor 7 結構保證	沒這概念	★ 結構 + integration test 驗證

場景	只有 12-factor	只有 90%	兩者皆有
Refactor 風險	Factor 12 純函數性幫助	測試保底	★ 結構 + 測試
Cross-team handoff	架構文件化	Test 是文件	★ 雙重文件

給工程 leadership 的綜合建議

1. 不要選邊站：兩條一起做。如果預算只夠做一條，先做 **90%**（短期 **ROI** 更高），同時規劃 **12-factor** 的 **phase-in**。

2. **Headcount** 配置：

- 50% 開發團隊（寫 feature with AI）
- 30% 平台 / quality team（負責 12-factor architecture + 90% gating）
- 20% reliability / SRE（production verification）

3. 季度 **OKR** 配對：

- Q1: 12-factor adoption（agent code 重構）+ Coverage 65% → 75%
- Q2: Factor 5/6/7 完成 + Coverage 75% → 85%
- Q3: Factor 9/10 完成 + Coverage 85% → 90% + Mutation 75%
- Q4: Audit + 升級到 Type D

4. 預算 **ratio**：

- AI tool 預算（包含寫測試 + 寫 feature）：\$2-5K/工程師/年
 - Verification infra：\$1-3K/工程師/年
 - 加起來 < 5% 工程薪資總和
-

終極論點

Garry Tan 跟 **Dex Horthy** 是同一場戰役的兩個將軍：

- Dex 守住「**LLM** 應用怎麼建」
- Garry 守住「**LLM** 應用怎麼驗」

兩條都贏，你的組織就贏。任何只贏一條的組織，最終會被同時做到兩條的對手取代。

“The companies that win the next 24 months won’t be the ones with the fastest agents...They’ll be the ones with the strongest guardrails.” — Garry Tan

把 12-factor 當「建造 guardrail 的圖紙」，90% 當「guardrail 的承重測試」。兩個都做完，你才真正擁有 **guardrail**。

結尾

恭喜你讀完整份 Garry Tan 複雜度棘輪 + 90% 測試覆蓋率精解。

一句話帶走

AI agent 寫程式比人類快 **10 倍**。如果驗證能力沒同步 **×10**，你就被複雜度棘輪追上了。**90%** 測試覆蓋率（含 **mutation $\geq 75\%$** ）是 **AI** 時代的新基線。

該做的第一步

1. 跑 coverage report，看你現在數字
2. 看 [Ch08 路線圖](#) 的 Week 1
3. 設一個 12 週後的 90% 目標
4. 開始

[← 回 README](#)